

GradeSense Source Code Documentation

Curated, selectable implementation excerpts from the production project

Curated, implementation-linked documentation. Code examples are selectable text, never screenshots.

Candidate

Name: Bhavuk Mahajan

Candidate ID: 20484629



1. Repository Structure and Execution Path

```
GradeSense/  
├── backend/app/api/routes/      # HTTP and WebSocket endpoints  
├── backend/app/services/      # intelligence and operations  
├── backend/app/models/        # SQLAlchemy persistence  
├── backend/app/schemas/      # Pydantic contracts  
├── frontend/src/              # React operator workspace  
├── data/                      # supplied datasets  
├── models/                   # immutable joblib artifacts  
└── render.yaml               # production blueprint
```



2. Backend Application Bootstrap

Purpose: validate model readiness, initialize governance, start streaming and register middleware/routes.

Inputs: environment-backed settings and immutable artifacts. Outputs: running FastAPI application and managed worker lifecycle.

```
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.api.router import api_router
from app.config.settings import get_settings
from app.core.exceptions import register_exception_handlers
from app.core.logging import configure_logging, get_logger
from app.database.session import SessionFactory, dispose_engine
from app.middleware import ProductionMiddleware
from app.services.intelligence import IntelligenceService
from app.services.operations import AuditService
from app.services.registry import ModelRegistryService
from app.services.streaming import get_streaming_service

settings = get_settings()
configure_logging(settings.log_level)
logger = get_logger(__name__)

@asynccontextmanager
async def lifespan(_ : FastAPI) -> AsyncIterator[None]:
    with SessionFactory() as session:
        IntelligenceService(settings, session).ensure_ready()
        ModelRegistryService().bootstrap_existing(session, settings)
        AuditService().record(session, "application_startup", "application")
    stream = get_streaming_service(settings)
    await stream.start()
    logger.info("application_started", extra={"environment": settings.environment})
    yield
    await stream.stop()
    with SessionFactory() as session:
        AuditService().record(session, "application_shutdown", "application")
    dispose_engine()
    logger.info("application_stopped")

def create_application() -> FastAPI:
    application = FastAPI(
        title=settings.app_name,
        version=settings.app_version,
        description="AI decision-support API for industrial paper grade transitions.",
        docs_url="/docs",
        redoc_url="/redoc",
        openapi_url="/openapi.json",
        lifespan=lifespan,
    )
```

3. Prediction API

Purpose: expose typed prediction and recommendation workflows without leaking service construction into route handlers.

```
router = APIRouter(tags=["intelligence"])

def get_intelligence_service(session: DatabaseSession) -> IntelligenceService:
    return IntelligenceService(get_settings(), session)

IntelligenceDependency = Annotated[IntelligenceService, Depends(get_intelligence_service)]

@router.post("/predict", response_model=PredictionResponse, summary="Predict transition quality")
def predict(process_input: ProcessInput, service: IntelligenceDependency) -> PredictionResponse:
    return service.predict(process_input)

@router.post(
    "/recommend",
    response_model=RecommendationResponse,
    summary="Generate model-evaluated corrective actions",
)
def recommend(
    process_input: ProcessInput, service: IntelligenceDependency
) -> RecommendationResponse:
    return service.recommend(process_input)

@router.get(
    "/correlations",
    response_model=CorrelationResponse,
    summary="Discover process correlations",
)
def correlations(
    service: IntelligenceDependency,
    limit: Annotated[int, Query(ge=1, le=50)] = 10,
) -> CorrelationResponse:
    return service.correlations(limit)

@router.get(
    "/dataset/statistics",
    response_model=DatasetStatisticsResponse,
    summary="Describe the generated industrial dataset",
)
def dataset_statistics(
    service: IntelligenceDependency,
) -> DatasetStatisticsResponse:
    return service.dataset_statistics()
```

4. Intelligence Orchestration

Purpose: coordinate preprocessing, inference, explainability, recommendation generation and persistence.

```
from app.services.model import ModelService
from app.services.recommendation import RecommendationService

class IntelligenceService:
    _training_lock = RLock()
    _dataset_lock = RLock()
    _dataset_cache: tuple[Path, int, pd.DataFrame] | None = None
    _statistics_cache: tuple[Path, int, DatasetStatisticsResponse] | None = None

    def __init__(self, settings: Settings, session: Session) -> None:
        self.settings = settings
        self.session = session
        self.dataset_generator = SyntheticDatasetGenerator()
        self.model_service = ModelService(settings)
        self.explainability_service = ExplainabilityService(self.model_service)
        self.recommendation_service = RecommendationService(self.model_service)

    def ensure_ready(self) -> None:
        if self.settings.model_path.exists() and self.settings.dataset_path.exists():
            self.model_service.latest_metadata(self.session)
            return
        missing = [
            str(path)
            for path in (self.settings.dataset_path, self.settings.model_path)
            if not path.exists()
        ]
        raise RuntimeError(
            "Required intelligence artifacts are unavailable; startup will not generate "
            f"or retrain them. Missing: {'', '.join(missing)}")

    def regenerate(self, records: int, seed: int) -> DatasetRegenerateResponse:
        with self._training_lock:
            frame = self.dataset_generator.generate(records, seed)
            self.dataset_generator.save(frame, self.settings.dataset_path)
            metadata = self.model_service.train(frame, self.session)
        return DatasetRegenerateResponse(
            records=len(frame),
            dataset_path=str(self.settings.dataset_path),
            model_version=metadata.version,
            training_metrics=metadata.metrics,
        )

    def predict(self, process_input: ProcessInput) -> PredictionResponse:
        values = process_input.model_dump()
        prediction = self.model_service.predict(values)
        explanation = self.explainability_service.explain(values, prediction)
        artifact = self.model_service.load()
        history = PredictionHistory(
            model_version=artifact["version"],
            input_data=values,
            quality_score=prediction.quality_score,
            off_spec_probability=prediction.off_spec_probability,
            stabilization_time=prediction.stabilization_time,
            explanation=explanation.model_dump(mode="json"),
        )
        self.session.add(history)
```

5. Forecast Model

Algorithm: transition-grouped validation, one HistGradientBoostingRegressor per horizon, GradientBoosting crossing classifier, and empirical residual intervals.

```

    random_state: int = 42,
) -> dict[str, Any]:
    unique_transitions = np.unique(windows.transition_ids)
    rng = np.random.default_rng(random_state)
    rng.shuffle(unique_transitions)
    split = max(1, int(len(unique_transitions) * 0.8))
    train_ids = set(unique_transitions[:split])
    train_mask = np.asarray(
        [transition_id in train_ids for transition_id in windows.transition_ids]
    )
    validation_mask = ~train_mask
    x_train = windows.features.loc[train_mask]
    x_validation = windows.features.loc[validation_mask]
    y_train = windows.targets[train_mask]
    y_validation = windows.targets[validation_mask]
    models: list[HistGradientBoostingRegressor] = []
    residual_intervals: list[float] = []
    horizon_mae: list[float] = []
    for horizon_index in range(forecast_horizon):
        model = HistGradientBoostingRegressor(
            learning_rate=0.07,
            max_iter=170,
            max_leaf_nodes=24,
            l2_regularization=0.2,
            random_state=random_state,
        )
        model.fit(x_train, y_train[:, horizon_index])
        validation_prediction = model.predict(x_validation)
        residuals = np.abs(y_validation[:, horizon_index] - validation_prediction)
        models.append(model)
        residual_intervals.append(float(np.quantile(residuals, 0.9)))
        horizon_mae.append(
            float(mean_absolute_error(y_validation[:, horizon_index], validation_prediction))
        )
    classifier = GradientBoostingClassifier(random_state=random_state)
    classifier.fit(x_train, windows.crossing[train_mask])
    crossing_probabilities = classifier.predict_proba(x_validation)[:, 1]
    crossing_auc = (
        float(roc_auc_score(windows.crossing[validation_mask], crossing_probabilities))
        if len(np.unique(windows.crossing[validation_mask])) > 1
        else 0.5
    )
    importance = permutation_importance(
        models[0],
        x_validation,
        y_validation[:, 0],
        n_repeats=3,
        random_state=random_state,
    )
    feature_importance = {
        feature: float(max(value, 0))
        for feature, value in zip(

```

6. Explainable AI

Algorithm: replace each input with its learned baseline, re-run the model, and rank the change in off-spec probability. This is local sensitivity, not causality.

```
class ExplainabilityService:
    """Local counterfactual attribution backed by the persisted prediction model."""

    def __init__(self, model_service: ModelService) -> None:
        self.model_service = model_service

    def explain(self, values: dict[str, Any], prediction: PredictionResult) -> Explanation:
        artifact = self.model_service.load()
        baselines = artifact["baselines"]
        global_importance = artifact["feature_importance"]
        contributions: list[FeatureContribution] = []
        for feature in PROCESS_FEATURES:
            counterfactual = dict(values)
            counterfactual[feature] = baselines[feature]
            baseline_prediction = self.model_service.predict(counterfactual)
            contribution = (
                prediction.off_spec_probability - baseline_prediction.off_spec_probability
            )
            contributions.append(
                FeatureContribution(
                    feature=feature,
                    value=values[feature],
                    contribution=round(contribution, 5),
                    importance=global_importance[feature],
                    direction="increases risk" if contribution >= 0 else "reduces risk",
                )
            )
        contributions.sort(
            key=lambda item: abs(item.contribution) * (0.5 + item.importance),
            reverse=True,
        )
        top = contributions[:5]
        summary = self._summary(top, prediction)
        return Explanation(
            summary=summary,
            top_contributing_features=top,
            feature_importance=global_importance,
        )

    @staticmethod
    def _summary(contributions: list[FeatureContribution], prediction: PredictionResult) -> str:
        risk_features = [item for item in contributions if item.contribution > 0]
        protective_features = [item for item in contributions if item.contribution <= 0]
        if risk_features:
            names = " and ".join(DISPLAY_NAMES[item.feature] for item in risk_features[:2])
            return (
                f"{names.capitalize()} are the strongest contributors to the "
                f"{prediction.off_spec_probability:.1%} predicted off-spec risk."
            )
        names = " and ".join(DISPLAY_NAMES[item.feature] for item in protective_features[:2])
        return (
            f"{names.capitalize()} currently reduce off-spec risk; the predicted probability is "
            f"{prediction.off_spec_probability:.1%}."
        )
```

7. Recommendation Engine

Purpose: generate bounded process interventions and keep only model-improving actions.

```
@dataclass(frozen=True)
class Intervention:
    variable: str
    candidate: Callable[[float], float]
    verb: str
    unit: str

INTERVENTIONS = [
    Intervention("steam_pressure", lambda value: value + 0.35, "Increase", "bar"),
    Intervention("steam_pressure", lambda value: value - 0.35, "Reduce", "bar"),
    Intervention("machine_speed", lambda value: value - 40, "Reduce", "m/min"),
    Intervention("dryer_temperature", lambda value: value + 4, "Increase", "°C"),
    Intervention("dryer_temperature", lambda value: value - 4, "Reduce", "°C"),
    Intervention("pulp_consistency", lambda value: value + 0.2, "Increase", "%"),
    Intervention("pulp_consistency", lambda value: value - 0.2, "Reduce", "%"),
    Intervention("reel_tension", lambda value: value - 0.3, "Reduce", "kN/m"),
    Intervention("refining_energy", lambda value: value + 10, "Increase", "kWh/t"),
    Intervention("headbox_pressure", lambda value: value - 0.2, "Reduce", "bar"),
]

class RecommendationService:
    def __init__(self, model_service: ModelService) -> None:
        self.model_service = model_service

    def recommend(self, values: dict[str, Any], baseline: PredictionResult) -> list[Recommendation]:
        evaluated: list[tuple[float, Intervention, float, PredictionResult]] = []
        for intervention in INTERVENTIONS:
            current = float(values[intervention.variable])
            lower, upper = PROCESS_RANGES[intervention.variable]
            candidate = min(max(intervention.candidate(current), lower), upper)
            if candidate == current:
                continue
            adjusted = dict(values)
            adjusted[intervention.variable] = candidate
            predicted = self.model_service.predict(adjusted)
            risk_improvement = baseline.off_spec_probability - predicted.off_spec_probability
            quality_improvement = predicted.quality_score - baseline.quality_score
            stabilization_improvement = baseline.stabilization_time - predicted.stabilization_time
            score = (
                risk_improvement
                + max(quality_improvement, 0) / 200
                + max(stabilization_improvement, 0) / 300
            )
            evaluated.append((score, intervention, candidate, predicted))

        evaluated.sort(key=lambda item: item[0], reverse=True)
        recommendations: list[Recommendation] = []
        used_variables: set[str] = set()
        for score, intervention, candidate, predicted in evaluated:
            if intervention.variable in used_variables or score <= 0:
                continue
            risk_points = max(
                (baseline.off_spec_probability - predicted.off_spec_probability) * 100,
                0,
            )
            quality_gain = max(predicted.quality_score - baseline.quality_score, 0)
            stabilization_gain = max(baseline.stabilization_time - predicted.stabilization_time, 0)
            expected_improvement = risk_points + quality_gain + stabilization_gain / 3
            if expected_improvement < 0.001:
                continue
            confidence = min(
                0.98,
                0.5 + min(abs(score) * 4, 0.3) + abs(baseline.off_spec_probability - 0.5) * 0.2,
            )
            recommendations.append(
                Recommendation(
                    text=(
```


8. Forecast-Backed Intervention Engine

Purpose: validate recipe/equipment constraints, simulate candidates through the active forecast model, rank improvements, and persist evidence.

```

)
from app.services.constraints import LIMITS, ConstraintEngine
from app.services.forecasting.service import ForecastingService

def recommendation_response(row: ForecastRecommendation) -> ForecastRecommendationResponse:
    explanation = dict(row.explanation)
    public_explanation = {
        key: explanation[key]
        for key in (
            "selection_reason",
            "forecast_causes",
            "trajectory_effect",
            "expected_risks",
            "remaining_uncertainty",
            "recipe_attribution",
        )
        if key in explanation
    }
    return ForecastRecommendationResponse(
        recommendation_id=row.id,
        forecast_id=row.forecast_id,
        state=row.state,
        rank=row.rank,
        affected_variables=row.affected_variables,
        current_values=explanation.get("current_values", {}),
        changes=row.changes,
        baseline_trajectory=row.baseline_trajectory,
        intervention_trajectory=row.intervention_trajectory,
        metrics=row.metrics,
        confidence=row.confidence,
        constraint_validation=row.constraint_validation,
        explanation=public_explanation,
        inference_sources=explanation.get("inference_sources", ["Forecast", "Historical Trend"]),
        historical_evidence=explanation.get(
            "historical_evidence",
            {
                "similar_transition_count": 0,
                "historical_acceptance_rate": 0,
                "historical_effectiveness": 0,
            },
        ),
        created_at=row.created_at,
        updated_at=row.updated_at,
        expires_at=row.expires_at,
    )

class InterventionEngine:
    def __init__(self, forecasting: ForecastingService) -> None:
        self.forecasting = forecasting
        self.constraints = ConstraintEngine()

```

9. Streaming and WebSocket

Purpose: broadcast bounded real-time sensor, prediction, alert and recommendation state to dashboards.

```

)
from app.services.alert import AlertService
from app.services.drift import DriftService
from app.services.forecasting.service import ForecastingService
from app.services.intelligence import IntelligenceService
from app.services.operations import AuditService, metrics_service

class ConnectionManager:
    def __init__(self) -> None:
        self.clients: set[WebSocket] = set()

    async def connect(self, websocket: WebSocket) -> None:
        await websocket.accept()
        self.clients.add(websocket)

    def disconnect(self, websocket: WebSocket) -> None:
        self.clients.discard(websocket)

    async def broadcast(self, event: str, data: Any) -> None:
        payload = {
            "event": event,
            "timestamp": datetime.now(UTC).isoformat(),
            "data": (data.model_dump(mode="json") if hasattr(data, "model_dump") else data),
        }
        stale: list[WebSocket] = []
        for client in tuple(self.clients):
            send_started = time.perf_counter()
            try:
                await client.send_json(payload)
                metrics_service.observe("websocket", time.perf_counter() - send_started)
            except Exception:
                metrics_service.observe("websocket", time.perf_counter() - send_started, True)
                stale.append(client)
        for client in stale:
            self.disconnect(client)

class StreamingService:
    def __init__(self, settings: Settings) -> None:
        self.settings = settings
        self.connections = ConnectionManager()
        self.status = "stopped"
        self.session_id: UUID | None = None
        self.started_at: datetime | None = None
        self.sample_count = 0
        self.latest_sample_at: datetime | None = None
        self.latest = LiveMetricsResponse(
            sensor=None,
            prediction=None,
            recommendations=[],
            alerts=[],
            drift=None,
            updated_at=None,
        )
        self._task: asyncio.Task[None] | None = None
        self._history: deque[dict[str, Any]] = deque(maxlen=720)
        self._recent_samples: deque[ProcessInput] = deque(maxlen=120)
        self._sequence_history: deque[dict[str, Any]] = deque(
            maxlen=settings.forecast_history_window
        )
        self._prediction_risks: deque[float] = deque(maxlen=120)

```

10. Database Session Management

Purpose: configure SQLite for development and pooled PostgreSQL for deployment using the same SQLAlchemy session contract.

```
from collections.abc import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker

from app.config.settings import get_settings
from app.database.base import Base

settings = get_settings()
is_sqlite = settings.database_url.startswith("sqlite")
connect_args = (
    {"check_same_thread": False}
    if is_sqlite
    else {"connect_timeout": settings.database_connect_timeout_seconds}
)
pool_options = (
    {}
    if is_sqlite
    else {
        "pool_size": settings.database_pool_size,
        "max_overflow": settings.database_max_overflow,
        "pool_timeout": settings.database_pool_timeout_seconds,
        "pool_recycle": settings.database_pool_recycle_seconds,
    }
)
engine = create_engine(
    settings.database_url,
    pool_pre_ping=True,
    connect_args=connect_args,
    **pool_options,
)
SessionFactory = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

def get_db_session() -> Generator[Session]:
    session = SessionFactory()
    try:
        yield session
    finally:
        session.close()

def dispose_engine() -> None:
```

11. Production Middleware

Purpose: request IDs, payload limits, rate limiting, latency/error metrics, audit support and security headers.

```
from app.services.operations import AuditService, metrics_service

class ProductionMiddleware(BaseHTTPMiddleware):
    def __init__(
        self,
        app,
        max_payload_bytes: int = 2_000_000,
        rate_limit_per_minute: int = 300,
    ) -> None:
        super().__init__(app)
        self.max_payload_bytes = max_payload_bytes
        self.rate_limit = rate_limit_per_minute
        self.clients: dict[str, deque[float]] = defaultdict(deque)
        self._rate_lock = RLock()

    async def dispatch(self, request: Request, call_next):
        request_id = request.headers.get("X-Request-ID") or str(uuid4())
        request.state.request_id = request_id
        try:
            content_length = int(request.headers.get("content-length", "")) or 0
        except ValueError:
            return self._secured(
                JSONResponse(
                    status_code=400,
                    content={"detail": "Invalid Content-Length header."},
                ),
                request_id,
            )
        if content_length > self.max_payload_bytes:
            return self._secured(
                JSONResponse(status_code=413, content={"detail": "Payload too large."}),
                request_id,
            )
        client = request.client.host if request.client else "unknown"
        now = time.monotonic()
        with self._rate_lock:
            recent = self.clients[client]
            while recent and recent[0] < now - 60:
                recent.popleft()
            if len(recent) >= self.rate_limit:
                return self._secured(
                    JSONResponse(
                        status_code=429,
                        content={"detail": "Rate limit exceeded."},
                    ),
                    request_id,
                )
            recent.append(now)
            if len(self.clients) > 10_000:
                self.clients = defaultdict(
                    deque,
                    {
                        key: values
                        for key, values in self.clients.items()
                        if values and values[-1] >= now - 60
                    },
                )
        started = time.perf_counter()
        metrics_service.request_started()
        try:
            response = await call_next(request)
```

12. Frontend Routing

Purpose: lazy-load operator and administrative pages within the shared application layout.

```
import { lazy, Suspense } from 'react'
import { Navigate, Route, Routes } from 'react-router'

import { AppLayout } from '../components/layout/AppLayout'
import { SkeletonGrid } from '../components/ui/QueryState'

const AnalyticsPage = lazy(() =>
  import('../pages/AnalyticsPage').then((module) => ({ default: module.AnalyticsPage })),
)
const CorrelationsPage = lazy(() =>
  import('../pages/CorrelationsPage').then((module) => ({ default: module.CorrelationsPage })),
)
const DashboardPage = lazy(() =>
  import('../pages/DashboardPage').then((module) => ({ default: module.DashboardPage })),
)
const DemoWorkspacePage = lazy(() =>
  import('../pages/DemoWorkspacePage').then((module) => ({
    default: module.DemoWorkspacePage,
  })),
)
const HistoryPage = lazy(() =>
  import('../pages/HistoryPage').then((module) => ({ default: module.HistoryPage })),
)
const ModelPage = lazy(() =>
  import('../pages/ModelPage').then((module) => ({ default: module.ModelPage })),
)
const NotFoundPage = lazy(() =>
  import('../pages/NotFoundPage').then((module) => ({ default: module.NotFoundPage })),
)
const PredictionPage = lazy(() =>
  import('../pages/PredictionPage').then((module) => ({ default: module.PredictionPage })),
)
const RecommendationHistoryPage = lazy(() =>
  import('../pages/RecommendationHistoryPage').then((module) => ({
    default: module.RecommendationHistoryPage,
  })),
)
const RecommendationsPage = lazy(() =>
  import('../pages/RecommendationsPage').then((module) => ({
    default: module.RecommendationsPage,
  })),
)
const SimulatorPage = lazy(() =>
  import('../pages/SimulatorPage').then((module) => ({ default: module.SimulatorPage })),
)
const SystemStatusPage = lazy(() =>
  import('../pages/SystemStatusPage').then((module) => ({ default: module.SystemStatusPage })),
)
const ModelRegistryPage = lazy(() =>
  import('../pages/AdminPages').then((module) => ({ default: module.ModelRegistryPage })),
)
const SystemMetricsPage = lazy(() =>
  import('../pages/AdminPages').then((module) => ({ default: module.SystemMetricsPage })),
)
const AuditLogPage = lazy(() =>
  import('../pages/AdminPages').then((module) => ({ default: module.AuditLogPage })),
)
const ConfigurationPage = lazy(() =>
  import('../pages/AdminPages').then((module) => ({ default: module.ConfigurationPage })),
)
const HealthDashboardPage = lazy(() =>
  import('../pages/AdminPages').then((module) => ({ default: module.HealthDashboardPage })),
)
const ExportCenterPage = lazy(() =>
  import('../pages/AdminPages').then((module) => ({ default: module.ExportCenterPage })),
)

export function App() {
  return (
    <Suspense
```

13. Typed API Client

Purpose: centralize origin validation, JSON error handling and typed endpoint methods.

```

    RelationshipDiscovery,
    DemoSeedResult,
  } from './types'

const API_URL = (import.meta.env.VITE_API_URL ?? '').trim().replace(/\/$/, '')
if (!API_URL) {
  throw new Error('VITE_API_URL must be configured before building or starting GradeSense.')
}

export class ApiError extends Error {
  constructor(
    message: string,
    public status?: number,
    public details?: unknown,
  ) {
    super(message)
  }
}

async function request<T>(
  path: string,
  options?: RequestInit,
  timeoutMs = 30_000,
): Promise<T> {
  const controller = new AbortController()
  const timeout = window.setTimeout(() => controller.abort(), timeoutMs)
  try {
    const response = await fetch(`${API_URL}${path}`, {
      ...options,
      headers: { 'Content-Type': 'application/json', ...options?.headers },
      signal: controller.signal,
    })
    const payload = await response.json().catch(() => null)
    if (!response.ok) {
      const message =
        payload?.error?.message ||
        (typeof payload?.detail === 'string' ? payload.detail : undefined) ||
        payload?.detail?.[0]?.msg ||
        `Request failed (${response.status})`
      throw new ApiError(message, response.status, payload)
    }
    return payload as T
  } catch (error) {
    if (error instanceof ApiError) throw error
    if (
      (error instanceof DOMException && error.name === 'AbortError') ||
      (error instanceof Error && error.name === 'AbortError')
    ) {
      throw new ApiError(
        'This request timed out while the production service was starting. Please retry.',
        408,
      )
    }
    throw new ApiError('Unable to connect to the GradeSense backend.')
  } finally {
    window.clearTimeout(timeout)
  }
}

const post = <T>(path: string, body: ProcessInput) =>
  request<T>(path, { method: 'POST', body: JSON.stringify(body) })

export const api = {
  health: () => request<Health>('/health'),
  modelInfo: () => request<ModelInfo>('/model/info'),
  datasetStatistics: () => request<DatasetStatistics>('/dataset/statistics'),
  correlations: (limit = 20) => request<Correlations>(`/correlations?limit=${limit}`),
  relationshipDiscovery: (stage: 'early' | 'middle' | 'late', limit = 10) =>
    request<RelationshipDiscovery>(
      `/relationships/discovery?stage=${stage}&min_strength=0.1&limit=${limit}`,
      undefined,
      60_000,
    )
}

```

14. Live Dashboard Hook

Purpose: maintain connection state, merge WebSocket events and expose trend-ready values to React components.

```

    stabilization: number
    steamPressure: number
    moisture: number
    temperature: number
    machineSpeed: number
  }

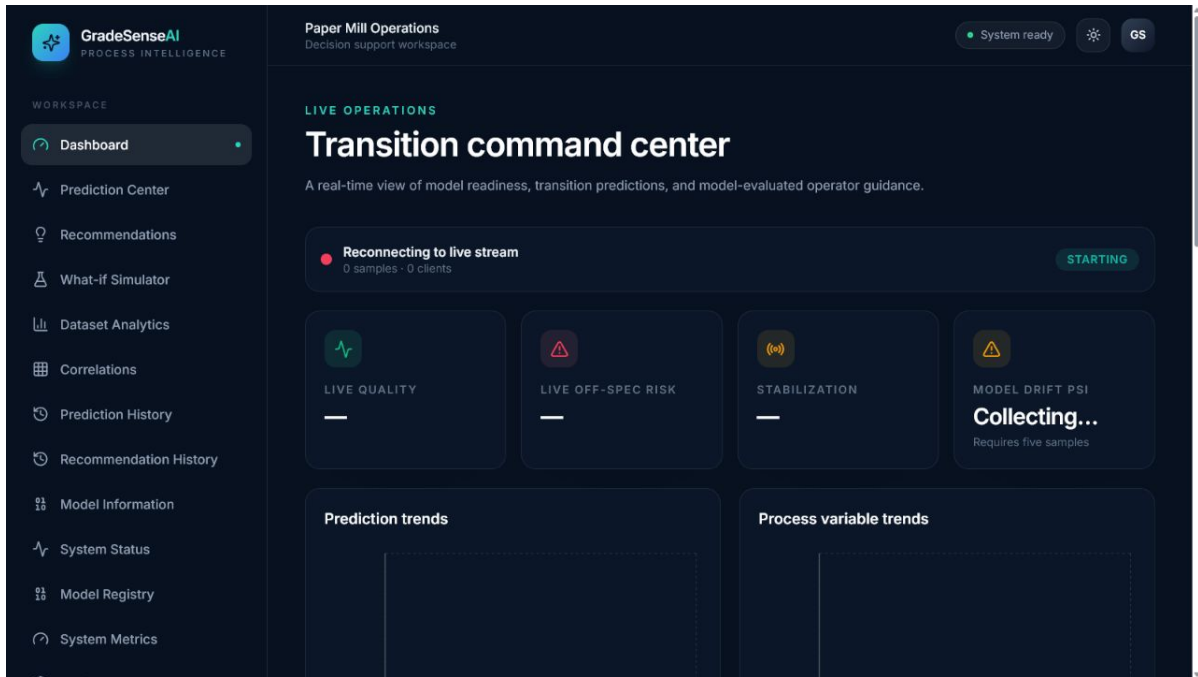
export function useLiveStream() {
  const [connected, setConnected] = useState(false)
  const [live, setLive] = useState<LiveMetrics>({
    sensor: null,
    prediction: null,
    recommendations: [],
    alerts: [],
    drift: null,
    updated_at: null,
  })
  const [status, setStatus] = useState<StreamStatus>()
  const [trends, setTrends] = useState<TrendPoint[]>([])
  const [forecast, setForecast] = useState<Forecast>()
  const reconnect = useRef<number | undefined>(undefined)
  const sensor = useRef<ProcessInput | null>(null)

  useEffect(() => {
    let active = true
    let socket: WebSocket | undefined
    const hydrate = async () => {
      const [metrics, stream] = await Promise.allSettled([api.liveMetrics(), api.streamStatus()])
      if (active) {
        if (metrics.status === 'fulfilled') {
          setLive(metrics.value)
          sensor.current = metrics.value.sensor
        }
        if (stream.status === 'fulfilled') setStatus(stream.value)
      }
    }
    void hydrate().catch(() => undefined)

    const connect = () => {
      socket = new WebSocket(websocketUrl())
      socket.onopen = () => setConnected(true)
      socket.onclose = () => {
        setConnected(false)
        if (active) reconnect.current = window.setTimeout(connect, 2000)
      }
      socket.onmessage = (message) => {
        const event = JSON.parse(message.data) as { event: string; data: unknown }
        if (event.event === 'sensor_update') {
          sensor.current = event.data as ProcessInput
          setLive((current) => ({ ...current, sensor: sensor.current }))
        } else if (event.event === 'prediction') {
          const prediction = event.data as Prediction
          setLive((current) => ({
            ...current,
            prediction,
          })

```

15. Frontend Visual Architecture



Module	Responsibility
App.tsx	Lazy routes and shell
api/client.ts	Typed REST and WebSocket origins
useLiveStream.ts	Reconnect and event state
DashboardPage.tsx	Operational summary
DemoWorkspacePage.tsx	End-to-end decision workflow
Prediction/Recommendation pages	Typed process forms and inference results

16. Deployment Blueprint

Purpose: define the production API and static frontend without embedding credentials.

```
services:
- type: web
  name: gradesense-api
  runtime: python
  plan: free
  buildCommand: >
    pip install --upgrade pip &&
    pip install ./backend &&
    cd backend &&
    export GRADESENSE_DATABASE_URL="$DATABASE_URL" &&
    alembic upgrade head
  startCommand: >
    cd backend &&
    export GRADESENSE_DATABASE_URL="$DATABASE_URL" &&
    export GRADESENSE_CORS_ORIGINS=["\"https://gradesense-4web.onrender.com\""] &&
    exec uvicorn app.main:app --host 0.0.0.0 --port "$PORT"
  healthCheckPath: /health
  maxShutdownDelaySeconds: 30
  autoDeployTrigger: commit
  envVars:
    - key: PYTHON_VERSION
      value: 3.13.5
    - key: GRADESENSE_ENVIRONMENT
      value: production
    - key: GRADESENSE_LOG_LEVEL
      value: INFO
    - key: DATABASE_URL
      sync: false

- type: web
  name: gradesense
  runtime: static
  buildCommand: cd frontend && npm ci && npm run build
  staticPublishPath: ./frontend/dist
  autoDeployTrigger: commit

  envVars:
    - key: VITE_API_URL
      value: https://gradesense-api.onrender.com

  routes:
    - type: rewrite
      source: /*
      destination: /index.html
```

Security note

Authentication and RBAC are intentionally not implemented in the application. Production deployment must use an identity-aware gateway; recommendations remain advisory and never write directly to DCS/PLC equipment.